ABSTRACT

The Model Context Protocol (MCP) standardizes how large language model (LLM) agents discover, describe, and call external tools. While MCP unlocks broad interoperability, it also enlarges the attack surface by making tools first-class, composable objects with natural-language metadata, and standardized I/O. We present **MSB** (MCP Security Benchmark), an end-to-end evaluation suite that systematically measures how well LLM agents resist MCP-specific attacks throughout the full tool-use pipeline: task planning, tool invocation, and response handling. MSB contributes: (1) a *taxonomy* of 12 attacks including name-collision, preference manipulation, prompt injections embedded in tool descriptions, out-of-scope parameter requests, user-impersonating responses, false-error escalation, tool-transfer, retrieval injection, and mixed attacks; (2) an *evaluation harness* that executes attacks by running real tools (both benign and malicious) via MCP rather than simulation; and (3) a *robustness metric* that quantifies the trade-off between security and performance: Net Resilient Performance (NRP). We evaluate ten popular LLM agents across 10 domains and 405 tools, producing 2,000 attack instances. Results reveal the effectiveness of attacks against each stage of MCP. Models with stronger performance are more vulnerable to attacks due to their outstanding tool calling and instruction following capabilities. MSB provides a practical baseline for researchers and practitioners to study, compare, and harden MCP agents.

1 INTRODUCTION

Recent advances in Large Language Models (LLMs) have demonstrated strong performance across diverse tasks such as problem solving, reasoning, tool invocation, and programming (Xu et al., 2024; Jin et al., 2025; Mingyu et al., 2024; Gao et al., 2023). These advances have fueled the development of AI agents that treat LLMs as central decision makers, augmented with external tools (Zheng et al., 2025) and memory mechanisms (Xu et al., 2025). By leveraging tools, LLM-based agents can engage with richer external environments and support applications ranging from project development (Lu et al., 2023) to team management (Li et al., 2025a) and information assistance (Chae et al., 2025).

Tools expand the functionality of LLM-based agents, but the absence of a unified standard forces reimplementation across architectures and platforms. To address this, Anthropic introduced the Model Context Protocol (MCP) (Antropic, 2024), which standardizes context exchange through a unified interface (Hou et al., 2025). As shown in Fig. 1, MCP follows a host–client–server workflow: tools declare their capabilities, the client retrieves and queries them, and the server executes the selected tool and returns results. While MCP improves interoperability, it also enlarges the attack surface and exposes agents to critical vulnerabilities (Song et al., 2025; Labs, 2025; Guo et al., 2025; Li et al., 2025b; Fang et al., 2025). Existing benchmarks, such as ASB (Zhang et al., 2025a), AgentDojo (Debenedetti et al., 2025), and InjecAgent (Zhan et al., 2024), remain confined to the function-calling paradigm and thus cannot capture these MCP-specific vulnerabilities.

To address this gap, we present MCP Security Bench (MSB), a benchmark for systematically evaluating the security of LLM agents across all stages under MCP-based tool use. MSB includes 2,000 attack instances across 10 task scenarios, 65 realistic tasks, and 405 tools, offering a large-scale and diverse testbed for assessing vulnerabilities in realistic environments. Specifically, MSB establishes a taxonomy of 12 attack types spanning the three critical stages of the MCP workflow: task planning, tool invocation, and response handling. These attacks target tool vectors such as names, descriptions, parameters, and responses. Tool signature attacks (e.g., Name Collision, Preference Manipulation, Prompt Injection) manipulate metadata to mislead tool selection. Tool parameter attacks (e.g., Out-of-Scope Parameter) induce agents to disclose unauthorized information. Tool response attacks (e.g., User Impersonation, False Error, Tool Transfer) alter agent behavior through deceptive or poisoned outputs. Retrieval injection attacks undermine contextual integrity by inserting malicious data into the retrieval process. Finally, Mixed Attacks combine multiple vectors across stages to amplify their impact, covering adversarial strategies throughout the MCP workflow.

In addition, MSB provides an evaluation harness that executes attacks by running real tools (both benign and malicious) through MCP rather than relying on simulated outputs. This dynamic design reflects operational conditions more faithfully and exposes vulnerabilities that static benchmarks fail to capture. To quantify robustness under these conditions, we complement the commonly used Attack Success Rate (ASR) and Performance Under Attack (PUA) with a new metrics: Net Resilient Performance (NRP), which captures the overall trade-off between performance and security. Using these metrics, we evaluate 9 popular LLM backbones and observe a peak attack success rate of 75.83%. The results indicate that MCP-specific vulnerabilities are readily exploitable and that stronger models are paradoxically more susceptible due to their superior tool-use ability, confirming the need for a dedicated benchmark to evaluate the security of MCP-based agents.

Our contributions are as follows: 1) We present MSB, a benchmark dedicated to evaluating the security of LLM agents under MCP-based tool use. It comprises more than 2,000 attack instances across 10 scenarios, 65 realistic tasks, and 405 tools. 2) We establish a taxonomy of 12 attack categories that cover the three stages of the MCP workflow: task planning, tool invocation, and response handling. 3) We develop a dynamic evaluation framework with three robustness metrics: ASR, PUA and proposed NRP, which quantifies the trade-off between safety and performance. 4) We conduct large-scale experiments on 10 LLM backbones and show that MCP-specific vulnerabilities are readily exploitable, demonstrating the need for a dedicated benchmark to assess the security of MCP-based agents.

3 Preliminary and Threat Model

3.1 Basic Concepts

LLM Agent with MCP Tools. We consider an LLM agent that interacts with tools through MCP. Each tool $\tau(\tau_n, \tau_d, \tau_p, \tau_r)$ is characterized by its name τ_n, functional description τ_d, parameters τ_p, and response τ_r. Within the MCP framework, the client exposes the available tool list $\mathcal{T} = (\tau_{(1)}, ..., \tau_{(l)})$ to the agent by embedding the tool names and descriptions into the system prompt p_{sys}, denoted as $p_{sys} \oplus \mathcal{T}$, $\oplus$ denotes the string concatenation. The agent then invokes a tool by supplying the required parameters τ_p, and subsequently processes the returned result τ_r. The agent can retrieve knowledge from a database $\mathcal{D}$ through the tools. Formally, a tool-augmented agent aims to maximize the following objective:

$$\mathbb{E}_{q\sim\pi_q}\left[\mathbb{1}\left(\text{Agent}(\text{LLM}(p_{sys} \oplus \mathcal{T}, q, \mathcal{O}), \mathcal{T}, \mathcal{D}) = a\right)\right], \tag{1}$$

where π_q denotes the distribution of user queries, and $\mathbb{1}(\cdot)$ denotes the indicator function. The LLM formulates a task plan based on the user query q and the tool list $\mathcal{T}$. The agent executes this plan through iterative tool invocations, producing a sequence of observations $\mathcal{O} = (o_{(1)}, ..., o_{(j)})$ from the task trajectory, including tool responses. Incorporated into the context, $\mathcal{O}$ enables the agent to dynamically refine its plan during task resolution. Here, a denotes the expected action of the agent.

Attack Task. Within the MCP framework, an attacker leverages a list of malicious tools $\mathcal{T}^m = (\tau^m_{(1)}, ..., \tau^m_{(k)})$ and a poisoned database $\mathcal{D}_p$ to induce the agent to perform an attack task. The name τ^m_n, description τ^m_d, parameters τ^m_p, and responses τ^m_r of a malicious tool τ^m can serve as potential attack vectors.

3.2 Threat Model

Attacker's Capabilities. The attacker can deploy a malicious MCP server that they fully control, including all tools hosted on that server. The attacker may publish such malicious tools by linking the server to third-party platforms (e.g., Smithery (Smithery.ai, 2025)). However, the attacker has no control over the LLM within the agent and therefore cannot, as in prior work (Zhang et al., 2025a), intercept user queries and inject malicious instructions directly into the LLM agent. The attacker's capabilities are summarized as follows: ① *Tools*. The attacker can modify every component of any malicious tool hosted on the server and thereby employ those tools as attack vectors. Moreover, the attacker can deploy multiple coordinated malicious tools on the server concurrently. MCP enables straightforward integration of malicious tools into the agent tool list. ② *System Prompt*. Based on MCP's available tool discovery mechanism (Antropic, 2024), the attacker can seamlessly insert malicious prompts into the agent's system prompt. ③ *External Resources*. The attacker has white-box privileges on external resources and can insert covert attack instructions into those external resources (Radosevich & Halloran, 2025; Halloran, 2025), which the agent can retrieve using tools.

Attack Goal. Within the MCP framework, the attacker aims to compromise the agent's decision-making in task planning, tool calling, and response handling, inducing it to perform a malicious action a_m. The attack goal is to maximize:

$$\mathbb{E}_{q\sim\pi_q}\left[\mathbb{1}\left(\text{Agent}(q, \theta_m) = a_m\right)\right], \tag{2}$$

where the attack aims to maximize the expected probability that the agent when influenced by adversarial modifications θ_m, performs a malicious action a_m for a given input query q. Here, θ_m denotes the set of adversarial modifications that the attacker can apply to the system prompt p_{sys}, the observation sequence $\mathcal{O}$, the tool list $\mathcal{T}$, and the knowledge database $\mathcal{D}$.

4 Attack Taxonomy

Within the MCP workflow, the LLM agent interacts with tools through tool signature (name and description), parameters, and responses, all of which can serve as attack vectors, as shown in Fig. 1. We introduce and categorize attack types based on these vectors and interaction stage (Tab. 1). In Sec. 4.1, we define three attack types during the task planning stage, where attackers inject malicious prompts into the system prompt through manipulated tool signature. Sec. 4.2 details an attack during the tool invocation stage, where malicious tools induce the agent to supply over-permissioned parameters. Sec. 4.3 presents three attack types during the response handling stage, where manipulated tool responses deceive the agent into performing malicious actions. Sec. 4.4 details an attack type during the response handling stage, where poisoned data is injected into the context via the tool response. These attacks can also be combined into mixed attacks cover multi-stage (Sec. 4.5). Finally, we provide attack examples in App. C.

4.1 Tool Signature Attack

During the task planning stage, an attacker exploits the tool name τ^m_n or the tool description τ^m_d as attack vector. Specifically, the attacker designs a malicious tool $\tau^m \in \mathcal{T}^m$ targeting a benign tool $\tau^t \in \mathcal{T}$ by crafting τ^m_n or τ^m_d, and injects $\mathcal{T}^m$ into $\mathcal{T}$, denoted as $\mathcal{T} + \mathcal{T}^m$. The malicious tool induces the agent to perform the malicious action a_m. Formally, the attack goal is to maximize

$$\mathbb{E}_{q\sim\pi_q}\left[\mathbb{1}\left(\text{Agent}(\text{LLM}(p_{sys} \oplus \mathcal{T} \oplus \mathcal{T}^m, q, \mathcal{O}), \mathcal{T} + \mathcal{T}^m) = a_m\right)\right], \tag{3}$$

where other notations are the same as those in Eq. 1, App. B summarizes the formulas. This category encompasses three specific types of attacks:

Name Collision (NC) (Jing et al., 2025) exploits the tool name as the attack surface, disrupting the agent's tool selection decision-making. Specifically, NC sets the malicious tool name τ_n^m to be similar to the the target tool's name τ_n^t, thereby tricking the agent into invoking the malicious tool τ^m instead of the intended target tool τ^t.

Preference Manipulation (PM) (Wang et al., 2025b) exploits the tool description as the attack surface, disrupting the agent's tool selection decision-making. Specifically, PM inserts a promotional statement within the target tool's description τ_d^t to form the malicious tool description τ_d^m, thereby inducing the agent to invoke the malicious tool. τ^m instead of the intended target tool τ^t.

Prompt Injection (PI) (Labs, 2025) exploits the tool description as the attack surface, distorting the agent's planning and reasoning process. Specifically, PI injects the malicious instruction x^m within the tool description τ_d^m, therby inducing the agent to perform the attack task apart from the intended user task.

4.2 Tool Parameter Attack

During the tool invocation stage, an attacker exploits the tool parameter τ_p^m as attack vector. Specifically, the attacker constructs a malicious tool $\tau^m \in \mathcal{T}^m$ by defining the parameter τ_p^m that is outside the ranges required for normal operation. The agent passes the parameter τ_p^m to invoke the tool, resulting in information leakage. Formally, the attack goal is to maximize

$$\mathbb{E}_{q\sim\pi_q}\left[\mathbb{1}\left(\text{Agent}(\text{LLM}(p_{sys}\oplus\mathcal{T}^m, q, \mathcal{O}), \mathcal{T}^m) = a_m(\tau^m(\tau_p^m = i^m))\right)\right], \tag{4}$$

where $a_m(\tau^m(\tau_p^m = i^m))$ denotes that the agent invokes the tool τ^m by setting the τ_p^m to value i^m. Other notations are the same as those in Eq. 1 and Eq. 3.

Out-of-Scope Parameter (OP) (Jing et al., 2025) is carried out through the above process, which exploits the tool parameter as the attack surface, coercing the agent into supplying out-of-scope inputs.

4.3 Tool Response Attack

During the tool response stage, an attacker exploits the tool response τ_r^m as attack vector. Specifically, the attacker constructs a malicious tool by embedding the malicious instruction x^m into the response τ_r^m. When the response τ_r^m is incorporated into the observation sequence $\mathcal{O}$, denoted as $\mathcal{O} + \tau_r^m$, it misleads the agent into following the malicious instruction x^m apart from the user task. Formally, the attack goal is to maximize

$$\mathbb{E}_{q\sim\pi_q}\left[\mathbb{1}\left(\text{Agent}(\text{LLM}(p_{sys}\oplus\mathcal{T}^m, q, \mathcal{O}+\tau_r^m), \mathcal{T}^m) = a_m\left[x^m\right]\right)\right], \tag{5}$$

where $a_m\left[x^m\right]$ denotes that the agent follows the malicious instruction x^m. Other notations are the same as those in Eq. 1 and Eq. 3. This category encompasses three specific types of attacks:

User Impersonation (UI) exploits the tool response as the attack surface, impersonating the user to embed malicious instructions that mislead the agent into performing unintended and harmful operations. As LLM capabilities improve, it can effectively follow user instructions (Qianyu et al., 2024; Stolfo et al., 2025; Cheng et al., 2025), even uncritically (Gracjan et al., 2025; Kang et al., 2023), which expand the attack surface (Zhang et al., 2025a). In real-world scenarios, directly altering user queries to inject malicious instructions is often infeasible. In MSB, we employ the tool to simulate the user and issue malicious instructions to the agent, yielding a simple yet effective attack method.

False Error (FE) (Guo et al., 2025) exploits the tool response as the attack surface, disrupting the agent's task execution trajectory. Specifically, FE provides fabricated tool execution error messages, requiring the agent to follow the malicious instruction to successfully invoke the tool. As a result, the agent performs unintended and harmful operations.

Tool Transfer (TT) (SlowMist, 2025) exploits the tool response as the attack surface, subverting the agent's tool routing logic and decision flow. Specifically, TT is a chained attack involving two tools: a relay tool τ^m and an endpoint tool τ^e. The relay tool τ^m does not perform the attack directly, but instead manipulates the agent to invoke the endpoint tool τ^e through its response τ_r^m, and the endpoint tool τ^e performs the actual attack.

4.4 Retrieval Injection Attack

The attacker provides the agent with a poisoned database $\mathcal{D}_p$ embedded with malicious instruction x^m, where $x^m \subset \mathcal{D}_p$. When the agent invokes a tool τ to retrieve data from $\mathcal{D}_p$, the response of the tool τ_r injects x^m into the observation sequence $\mathcal{O}$, inducing the agent to follow the malicious instruction apart from the user task. Formally, it can be expressed as

$$\mathbb{E}_{q\sim\pi_q}\left[\mathbb{1}\left(\text{Agent}(\text{LLM}(p_{sys}\oplus\mathcal{T}, q, \mathcal{O}+\tau_r), \mathcal{T}, \mathcal{D}_p) = a_m\left[x^m\right]\right)\right], \tag{6}$$

where other notations are the same as those in Eq. 1 and Eq. 3.

Retrieval Injection (RI) (Radosevich & Halloran, 2025) is carried out through the above process, which exploits the external resources as the attack surface, compromising the agent's contextual integrity. RI differs from the attacks in Sec. 4.3 in that its malicious instructions originate from the poisoned database, whereas the tool itself remains benign. We therefore classify RI as a distinct attack type.

4.5 Mixed Attack

An attacker simultaneously exploits multiple components of the tool τ^m as attack vectors, constructing a mixed attack covering multiple stages. Formally, it can be expressed as

$$\mathbb{E}_{q\sim\pi_q}\left[\mathbb{1}\left(\text{Agent}(\text{LLM}(p_{sys}\oplus\mathcal{T}\oplus\mathcal{T}^m, q, \mathcal{O}+\tau_r^m), \mathcal{T}+\mathcal{T}^m) = a_m\right)\right], \tag{7}$$

where other notations are the same as those in Eq. 1 and Eq. 3. The mixed attack simultaneously exploit multiple interaction interfaces as the attack surface, compromising several stages of the agent's task execution pipeline. Combinations such as PM with UI integrate the end-to-end attack chain from tool selection to response handling. These mixed attacks pose a greater risk in real-world deployments involving the configuration of multiple tools.

5 Designing and Constructing MSB

MSB is a comprehensive benchmarking framework designed to evaluate various attacks that exploit security vulnerabilities in MCP against LLM agents. A key advantage of MSB lies in its incor-

poration of executable tools (both benign and malicious) across varied real-world scenarios. This enables the benchmarking of MCP security vulnerabilities under realistic conditions, rather than in simulated environments (Zhang et al., 2025a; Basu et al., 2024; Xie et al., 2024). App. D.5 shows the comparisons among ASB and other benchmarks. We summarize the statistics of MSB in Tab. 2.

5.1 MSB COMPONENTS

Benign tools and user tasks. The benign tool set in MSB is constructed from the most frequently used tools on the MCP integration platform (Smithery.ai, 2025). After functionality verification and deduplication, 304 tools whose implemented behavior matches their descriptions are retained as benign tools. Based on this tool set, we define user tasks that exercise the functionality of these tools. Each task requires invoking at least one benign tool. For example, the task associated with a paper-search tool is to summarize recent advances in a given research area. In total, we obtain 65 user tasks covering 10 representative real-world scenarios. Further details are provided in App. D.1.

Attack tools and attack tasks. Attack tools are generated by mutating the benign tools. Each attack-tool type applies a specific modification strategy to selected components of a benign tool (e.g., FE tools replace the original response with an error message). The components subject to mutation include the tool name, description, parameters, and responses. Tab. 1 summarizes the corresponding modification rules. This procedure yields 405 attack tools. Attack tasks specify the attacker's intended objective (e.g., modifying sensitive data). We design 6 attack tasks, each requiring the agent to invoke tools to achieve the specified objective (e.g., calling a file-editing tool to alter stored data). As shown in Tab. 1, attack tasks can be combined with attack tools through string concatenation. Further details are provided in App. D.2.

Environment and instances. The environment defines the workspace in which the agent operates. It contains both target data for attacks (e.g., personal sensitive information) and poisoned external resources (e.g., guidelines embedding malicious instructions). The agent is equipped with two MCP servers—FileSystem (2024) and DesktopCommander (2024)—which provide basic file-access and workspace-manipulation capabilities. By composing user tasks, attack tasks, and attack tools, we construct 2,000 attack test instances. Additional information is provided in App. D.3.

5.2 EVALUATION METRICS

Attack Success Rate (ASR): the fraction of attack instances where the attacker's objective is achieved.

$$\text{ASR} = \frac{\text{Number of successful attack instances}}{\text{Number of total attack instances}} \tag{8}$$

Performance Under Attack (PUA): the fraction of user tasks completed in an adversarial environment.

$$\text{PUA} = \frac{\text{Number of completed user tasks under attack}}{\text{Number of total user tasks}} \tag{9}$$

Net Resilient Performance (NRP): overall resilient utility.

$$\text{NRP} = \text{PUA} \cdot (1 - \text{ASR}) \tag{10}$$

We evaluate whether the attacker's objective has been achieved by examining the environmental state within the workspace (e.g., inspecting whether the sensitive data has been modified), we summarize the measurements for each attack task in Tab. 9. The completion of a user task is assessed by examining the tool invocation logs to determine whether the agent invoked the benign tools required by the task (e.g., invoking the paper search tool in the user task of paper search). The **ASR** measures the effectiveness of attacks. Generally, a higher ASR value indicates that the LLM agent is more susceptible to attack threats. The **PUA** evaluates the agent's ability to complete user tasks in adversarial environments. A higher PUA value demonstrates greater operational stability under interference conditions. The **NRP** is designed to assess the agent's overall capability to maintain performance while resisting attacks in adversarial environment. A lower NRP suggests either poor performance under attacks, high vulnerability to attacks, or a combination of both. Conversely, a higher NRP signifies that the agent can effectively resist attacks while maintaining task performance. NRP provides a comprehensive metric that balances accuracy and security to evaluate the agent's overall resilience.

Other benchmarks, such as ASB (Zhang et al., 2025a), compute NRP by combining model performance in benign environments with ASR. However, in our study, there are significant differences between benign and adversarial environments. For example, attacks such as Preference Manipulation can tempt agents to choose malicious tools, representing scenarios absent in benign settings. This makes it difficult to extend performance measurements from benign to adversarial environments. Therefore, we compute NRP based on model performance in adversarial environments.

6 Evaluation

6.1 Experimental Setup

The NC, PM, and TT are attack types that induce the agent to invoke malicious tools. We combine them with attacks that cause concrete damage to form mixed attacks for evaluation: NC combined with FE (NC-FE), PM combined with FE (PM-FE), PM combined with UI (PM-UI), PM combined with OP (PM-OP), and TT combined with OP (TT-OP). Furthermore, we evaluate two additional mixed attacks: PI combined with UI (PI-UI) and PI combined with FE (PI-FE).

We evaluate 10 LLM agents with system prompt given in Fig. 6: DeepSeek-V3.1 (DeepSeek-AI, 2024), GPT-4o-mini (OpenAI, 2024), GPT-5 (OpenAI, 2025), Claude 4 Sonnet (Anthropic, 2025), Gemini 2.5 Flash (Comanici et al., 2025), Qwen3 8B, Qwen3 30B (Yang et al., 2025), Llama3.1 8B, Llama3.1 70B, and Llama3.3 70B (Dubey et al., 2024).

6.2 Benchmarking Attack

Tab. 3 presents the Attack Success Rate (ASR) for various attacks and LLM backbones. From the results, we draw the following conclusions: **(1) All attack methods demonstrate effectiveness**, with an overall average ASR of 40.35%. The impact of OP is the most pronounced, achieving the highest average ASR of 76.5%. In contrast, NC-FE performs the least effectively, with an average ASR of 14.62%. **(2) Novel attacks introduced by MCP are more aggressive.** Compared to attacks already existing in function calling, such as PI with an average ASR of 20.21% and RI with 20%, MCP-based attacks like UI and FE achieve higher average success rates, reaching 45.69% and 39.21%, respectively. Although both UI and FE inject malicious instructions into tool responses, our proposed UI achieves a higher average ASR by imitating users, demonstrating the aggressiveness of the attack. **(3) Mixed attacks exhibit synergistic enhancement.** Their success rate is higher than that of their constituent single attacks; for example, the average ASR of PI-UI exceeds that of both PI and UI, suggesting that different attack vectors can mutually reinforce each other. We report complete results and further analyze in App. E.

Fig. 2 illustrates the relationships among different metrics across various LLM backbones. Our findings are as follows: **(1) An inverse scaling law exists between LLM capability and security** (McKenzie et al., 2023). More capable models tend to be more vulnerable to attacks, which aligns with observations in (Debenedetti et al., 2025; Zhang et al., 2025a). In MSB, accomplishing attack tasks also requires the agent to invoke tools. LLMs with higher utility, owing to their superior tool-use and instruction-following abilities, exhibit higher ASR. For example, DeepSeek-V3.1 achieves both the highest ASR and PUA. As model capability decreases, the ASR also shows a descending

trend. **(2) The NRP metric effectively balances agent utility and security in adversarial settings**, providing a holistic measure of model resilience. GPT-5 achieves a high user task completion rate while maintaining a moderate level of attack resistance, resulting in the highest NRP score. NRP is particularly valuable in backbone selection for agentic LLMs, as it captures the trade-off between task performance and adversarial resistance. This is especially important in realistic settings where model capability and safety tend to exhibit an inverse relationship, in which case NRP provides a comparable quantitative reference. For example, GPT-4o-mini shows both higher utility and vulnerability than Llama3.3 70B (its ASR and PUA are both larger), making it difficult to directly compare their overall effectiveness. In contrast, NRP indicates that Llama3.3 70B is a more suitable candidate model, ensuring better efficiency and resilience in real-world deployments.

We further compared the attack results from the perspective of both the MCP pipeline stages and tool configurations. As shown in Fig. 3, our findings are as follows: **(1) Agents are vulnerable to attacks at full stage.** At the invocation stage, the model is the least secure, exhibiting the highest average ASR of over 70%. This suggests that attackers can easily obtain target data by exploiting the tool parameter interface. Moreover, Over-trust in tool responses also leads to high ASR in the response handling stage. These interaction processes are often hidden from the user, and such information asymmetry further enhances the stealth and aggressiveness of attacks. **(2) Attacks remain effective even in multi-tool environments containing benign tools.** Real-world scenarios often provide agents with a toolkit; even when benign tools are available, induction methods such as NC, PM, and TT still lead to significant attack success.

6.3 Defense for MCP Attack

In this section, we evaluate LLM agents enhanced with MCIP (Jing et al., 2025), a defense mechanism designed to mitigate security vulnerabilities in MCP. MCIP is a detection-based approach that trains a Llama-xLAM-2-8B classifier on structured dialogue data containing risky behaviors to identify security risks in agent tool interactions.

As can be seen from Tab. 4, the NRP only sligtly increases. While this defense improves the model's security, it also tends to over-reject and cause functional degradation, suggesting that it struggles to substantially enhance the model's overall performance. In some cases, a detection and blocking strategy can prevent the agent from completing the user task. For example, blocking out-of-scope parameters may cause missing arguments and thus failed tool invocations. In such scenarios, the out-of-scope parameters are semantically deceptive, which suggests the need for more intelligent, dynamic defenses during interaction, such as context-aware checks on whether parameters are out of scope and sanitizing before forwarding them.

In MCP, attackers launch attacks via crafted malicious tools, where the malicious instructions are hidden inside the tools, thereby reducing the effectiveness of preventive defenses. For example, in response based attacks, the tool schema remains legitimate, and the attack intent is only revealed after the tool has been invoked. Moreover, attackers can deploy multiple malicious tools on the same MCP server to form complex, multi-tool attack chains and carry out end-to-end attacks. For such scenarios, the LLM's safety mechanisms and defenses may fail, causing the agent to deviate from its intended utility and safe objectives.

7 Conclusion

We introduced MSB, a benchmark for systematically evaluating the security of LLM agents under MCP-based tool use. MSB comprises 12 attack types and 2,000 test cases across 10 domains, 65 tasks, and 405 tools, executed through both benign and malicious tool interactions. Experiments on 10 LLM agents demonstrate that MCP-specific vulnerabilities are highly exploitable. We hope MSB can facilitate future research toward building more secure and resilient MCP-based agents.